Exercise : Consumer Groups

Creating Kafka Topics

In this exercise, the user will learn to create topics using Command Line Interface(CLI).

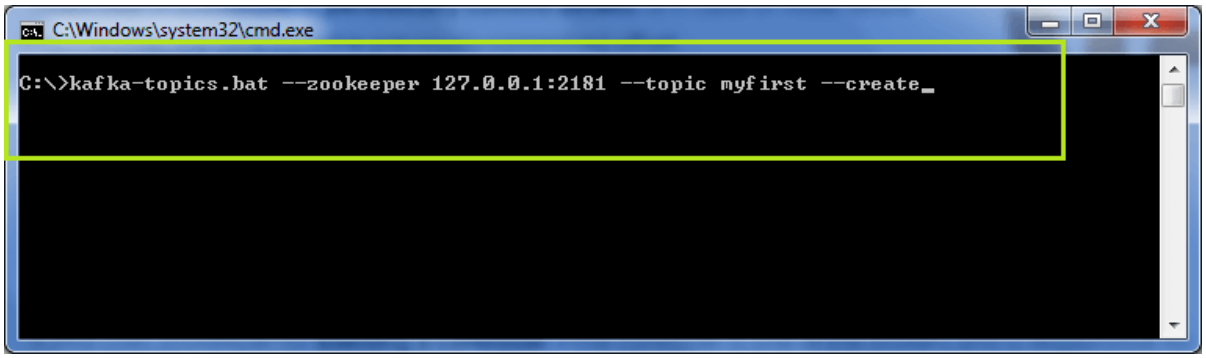
There are following steps used to create a topic:

Step1: Initially, make sure that both zookeeper, as well as the Kafka server, should be started.

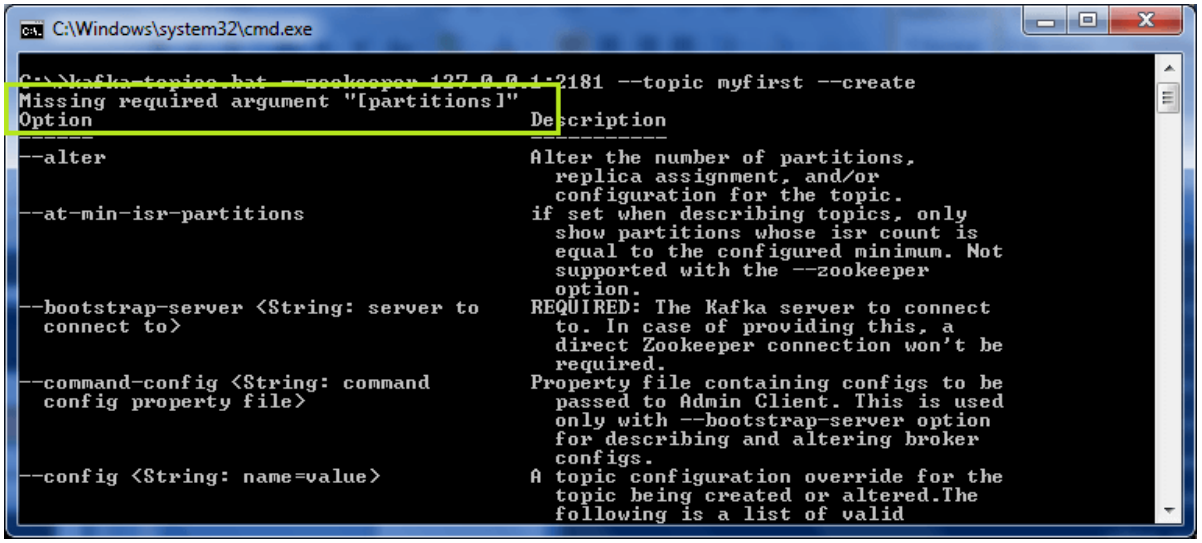
Step2: Type '**kafka-topics -zookeeper localhost:2181 -topic -create**' on the console and press enter. Here, the name of the topic is 'myfirst'

1. kafka-topics -zookeeper localhost:2181 -topic <topic_name> -create

Note: Instead '.bat', use '.sh' while creating topics on Linux(accordingly).



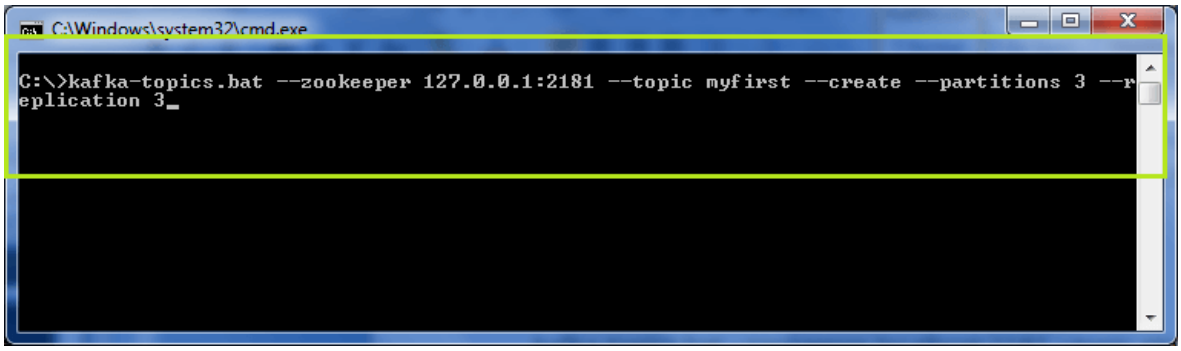
The output will be displayed as:



The above snapshot displays an error "**missing required argument partitions**". As it is necessary to declare the number of partitions and its ISR(replication factor), while creating a topic. If not declared, such type of errors will be thrown.

Step3: Now, rewrite the above command after fulfilling the necessities, as:

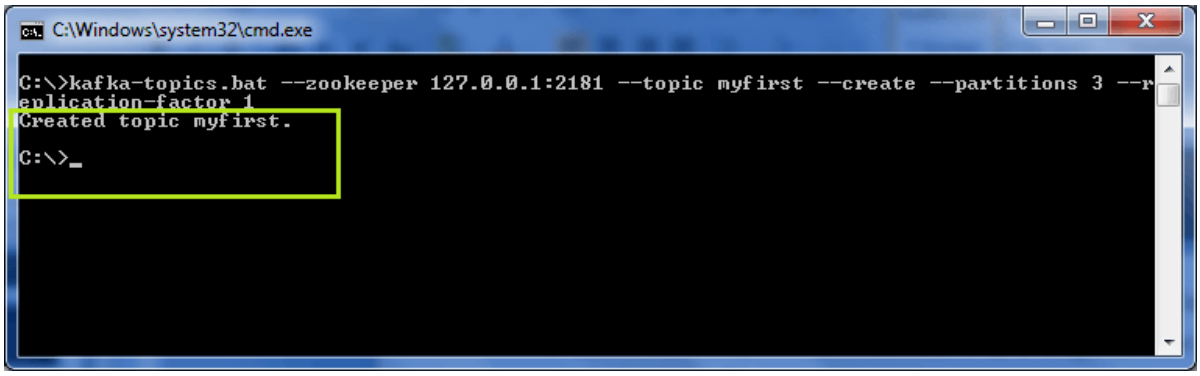
'**kafka-topics.bat -zookeeper localhost:2181 -topic <topic_name> --create --partitions <value> --replication-factor <value>**'. Press enter.



Again, an error will be thrown, indicating a replication factor greater than the number of brokers. Till now, we have started only a single broker, not multiple brokers. Thus, such type of error will appear.

Note: The replication factor can never be greater than the number of available brokers.

Step4: Consider the replication factor as 1 for now, and rewrite the above command. Press enter. The output will be displayed as:



Thus, if all the steps are processed successfully, then the topic will be created successfully, as shown in the above output.

Topic already exists

In case the user creates another topic with the same name as the existing topic, an error " **topic <topic_name> already exists**" will be thrown.

```
C:\Windows\system32\cmd.exe
C:\>kafka-topics.bat --zookeeper 127.0.0.1:2181 --topic myfirst --create --partitions 3 --r
eplication-factor 1
Error while executing topic command : Topic 'myfirst' already exists.
[2019-11-01 12:23:54,656] ERROR org.apache.kafka.common.errors.TopicExistsException: Topic
'myfirst' already exists.
 (kafka.admin.TopicCommand$)

C:\>_
```

Few more about topics

1) Listing the number of Topics

To list the number of topics created within a broker, use '**-list**' command as:

'kafka-topics.bat -zookeeper localhost:2181 -list'.

```
C:\Windows\system32\cmd.exe

C:\>kafka-topics.bat --zookeeper 127.0.0.1:2181 --list
myfirst
mysecond

C:\>_
```

There are two topics 'myfirst' and 'mysecond' present in the above snapshot.

2) Describing a topic

To describe a topic within the broker, use '**--describe**' command as:

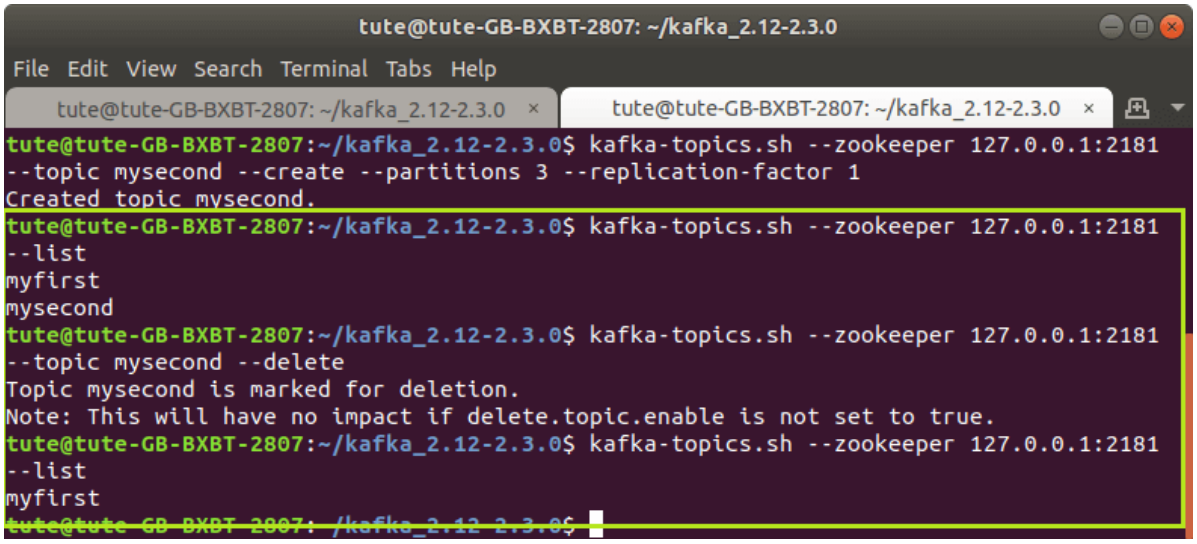
'kafka-topics.bat --zookeeper localhost:2181 --describe --topic <topic_name>'.
This command gives the whole description of a topic with the number of partitions, leader, replicas and, ISR.

```
C:\Windows\system32\cmd.exe

C:\>kafka-topics.bat --zookeeper 127.0.0.1:2181 --describe --topic myfirst
Topic:myfirst   PartitionCount:3        ReplicationFactor:1     Configs:
        Topic: myfirst  Partition: 0    Leader: 0       Replicas: 0     Isr: 0
        Topic: myfirst  Partition: 1    Leader: 0       Replicas: 0     Isr: 0
        Topic: myfirst  Partition: 2    Leader: 0       Replicas: 0     Isr: 0

C:\>_
```

3) Deleting a topic

To delete a topic, use '**-delete**' command. The delete command is used as: **'kafka-topics.sh -zookeeper localhost:2181 -topic<topic_name> --delete'**

> **Note:** Window users will face issues while deleting any topic. So, prefer this command on Linux, or on macOS.



In the above snapshot, it is seen that there were two existing topics 'myfirst' and 'mysecond'. After deleting 'mysecond' topic, only 'myfirst' topic is listed by the '**-list**' command.

So, in this way, various topics can be created and several commands can be applied on different topics.

Sending data to Kafka Topics

Kafka Console Producer

In order to send data to the Kafka topic, a producer is required. The role of the producer is to send or write data/messages to the Kafka topics.

In this section, we will learn how a producer sends messages to the Kafka topics.

There are following steps used to launch a producer:

Step1: Start the zookeeper as well as the kafka server.

Step2: Type the command: '**kafka-console-producer**' on the command line. This will help the user to read the data from the standard inputs and write it to the Kafka topic.

> **Note:** Choose '.bat' or '.sh' as per the operating system.

```
C:\>kafka-console-producer.bat
This tool helps to read data from standard input and publish it to Kafka.
Option                                   Description
------                                   -----------
--batch-size <Integer: size>             Number of messages to send in a single
                                           batch if they are not being sent
                                           synchronously. (default: 200)
--broker-list <String: broker-list>      REQUIRED: The broker list string in
                                           the form HOST1:PORT1,HOST2:PORT2.
--compression-codec [String:             The compression codec: either 'none',
  compression-codec]                       'gzip', 'snappy', 'lz4', or 'zstd'.
                                           If specified without value, then it
                                           defaults to 'gzip'
--help                                   Print usage information.
--line-reader <String: reader_class>     The class name of the class to use for
                                           reading lines from standard in. By
                                           default each line is read as a
                                           separate message. (default: kafka.
```

```
Select C:\Windows\system32\cmd.exe
                                           the producer waits before refreshing
                                           the metadata. (default: 100)
--socket-buffer-size <Integer: size>     The size of the tcp RECV size.
                                           (default: 102400)
--sync                                   If set message send requests to the
                                           brokers are synchronously, one at a
                                           time as they arrive.
--timeout <Integer: timeout_ms>          If set and the producer is running in
                                           asynchronous mode, this gives the
                                           maximum amount of time a message
                                           will queue awaiting sufficient batch
                                           size. The value is given in ms.
                                           (default: 1000)
--topic <String: topic>                  REQUIRED: The topic id to produce
                                           messages to.
--version                                Display Kafka version.

C:\>_
```

The highlighted text represents that a '**broker-list**' and a '**topic id**' is required to produce a message. It is because a producer must know the id of the topic to which the data is to be written.

Step3: After knowing all the requirements, try to produce a message to a topic using the command:

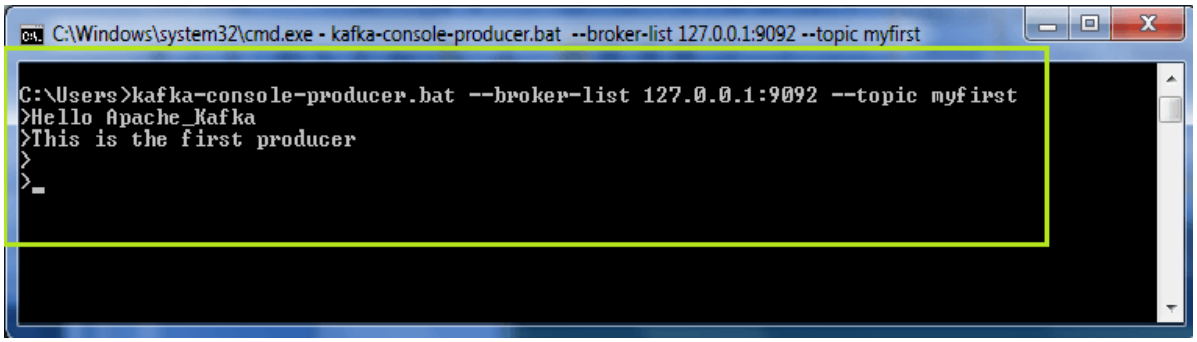
'kafka-console-producer -broker-list localhost:9092 -topic <topic_name>'. Press enter.

```
C:\Users>kafka-console-producer.bat --broker-list 127.0.0.1:9092 --topic myfirst
>
```

Note: Here, 9092 is the port number of the Kafka server.

Here, 'myfirst' topic is chosen to write messages to.

A '>' will appear in the new line. Start producing some messages, as shown below:



Step4: Press 'Ctrl+c' and exist by pressing the 'Y' key.

So, in this way, a producer can produce/send several messages to the Kafka topics.

Producer with Keys

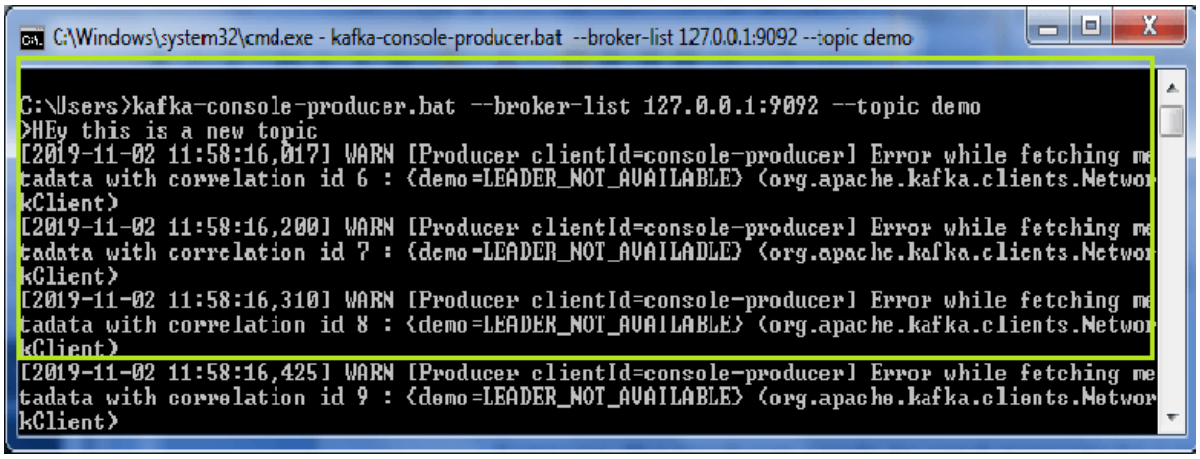
A Kafka producer can write data to the topic either with or without a key. If a producer does not specify a key, the data will be stored to any of the partitions with key=null, else the data will be stored to the specified partition only. A '**parse.key**' and a '**key.seperator**' is required to specify a key for the topic. The command used is:

1. 'kafka-console-producer --broker-list localhost:9092 --topic <topic_name> --property parse.key=**true** --property key.separator=,
2. > key,value
3. > another key,another value'

Here, key is the specific partition, and value is the message to be written by the producer to the topic.

When a topic does not exist?

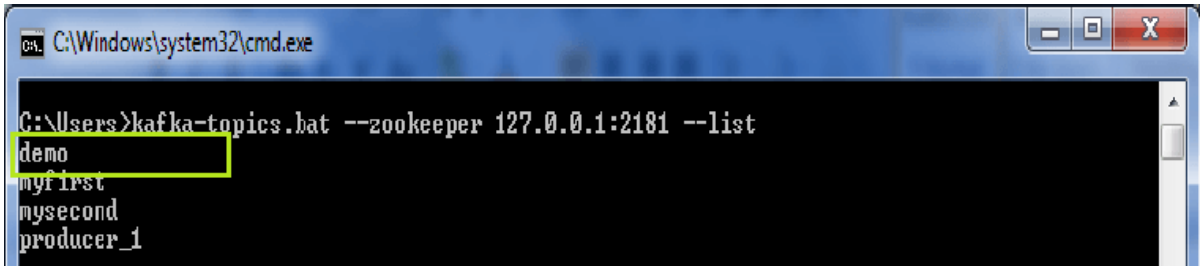
Suppose the producer wants to send messages to a new topic that does not exist yet. In such a situation, a warning will appear, as shown in the below snapshot, after producing a message. It is just a warning.



Why this warning?

The warning occurred because earlier the topic 'demo' didn't exist. But, as soon the producer wrote a message, Kafka somehow created that topic. Although, no leader election held for this unexpected topic, '**LEADER_NOT_AVAILABLE**' error could be seen. But, for the next time, the producer can continue to write more messages as no warning will appear again. It is because the topic comes in the existing list now.

The users can check using the '**-list**' command, as shown below:

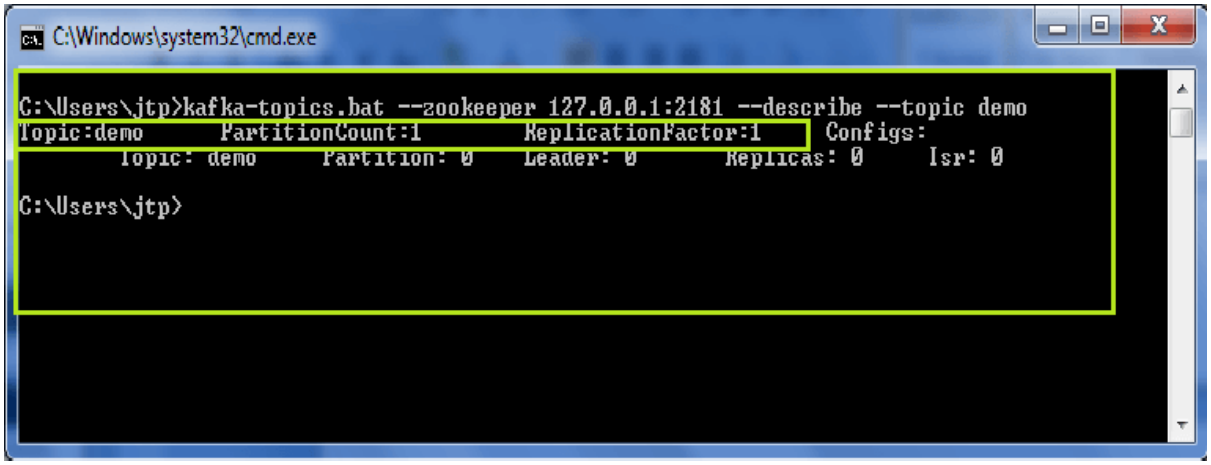


The topic 'demo' can be seen on the list.

Describing the new topic

As such topics which are created directly by the producer grabs the default number of partitions and its replication factor as 1.

For example,



The topic 'demo' when described using the '-**-describe**' command, gives the value of 'PartitionCount' and 'ReplicationFactor' as 1(default value). Thus, it is always a better option to create a topic before producing messages to it.

Changing the Default Values

Follow the below steps to change the default values for the new topic:

1. Open '**server.properties**' file using Notepad++, or any other text editor.

2. Edit the value of num.partitions=1 to a new value. Let it be 3. So, whenever such new topics are introduced, the number of PartitionCount and ReplicationFactor will be 3(whatever the user has set).
3. Save the file.

But, always create topics before.

Kafka Console Consumer

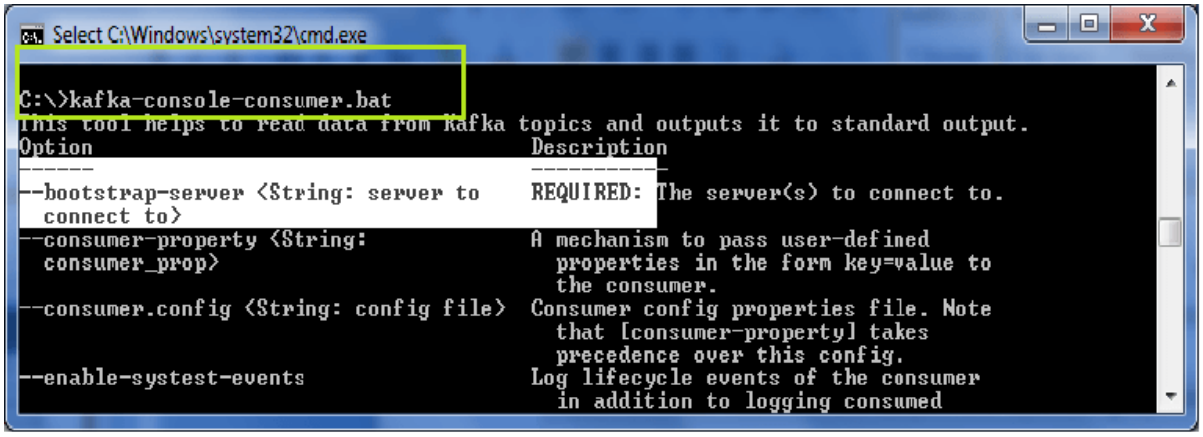
In this section, the users will learn how a consumer consumes or reads the messages from the Kafka topics.

There are following steps taken by the consumer to consume the messages from the topic:

Step 1: Start the zookeeper as well as the kafka server initially.

Step2: Type the command: '**kafka-console-consumer**' on the command line. This will help the user to read the data from the Kafka topic and output it to the standard outputs.

Note: Choose '.bat' or '.sh' as per the operating system.

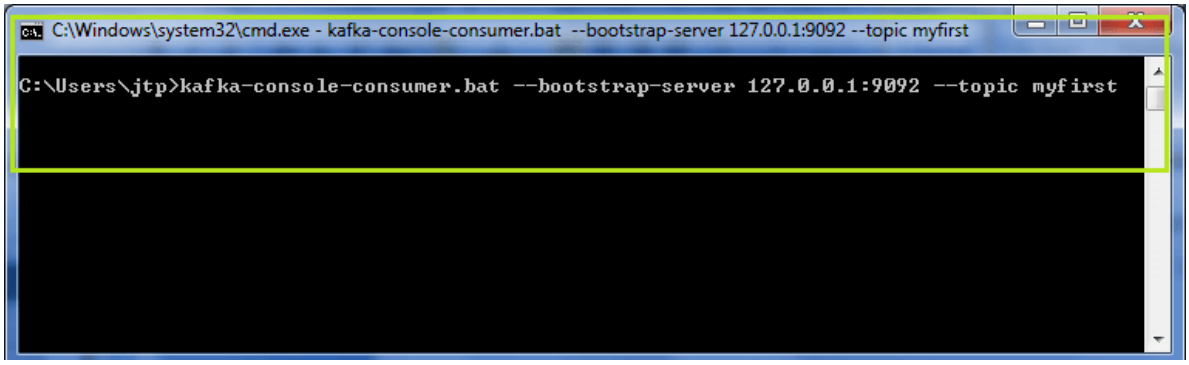


The highlighted text represents that a 'bootstrap-server' is required for the consumer to get connected to the Kafka topics. Also, a 'topic_id' is required to know from which topic the consumer will read the messages.

Step3: After knowing all the requirements, try to consume a message from a topic using the command:

'kafka-console-consumer --bootstrap-server localhost:9092 --topic <topic_name>'. Press enter.

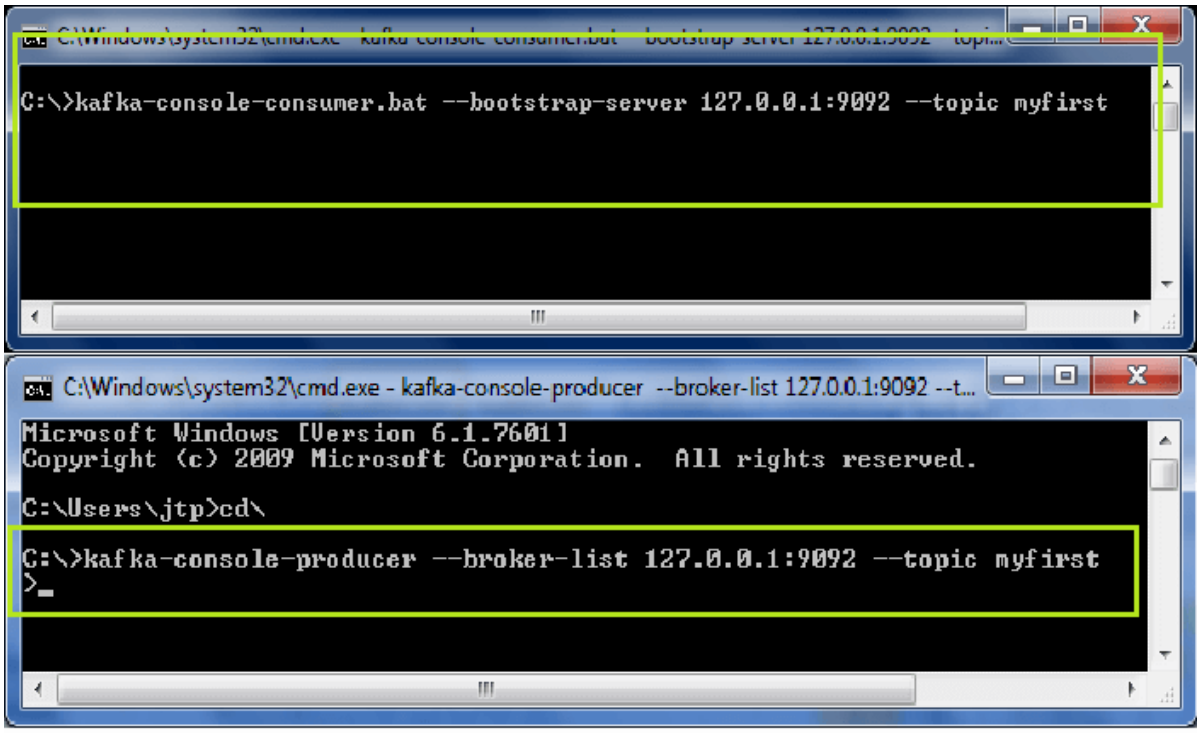
Note: Bootstrap server is the Kafka server, having port number=9092.



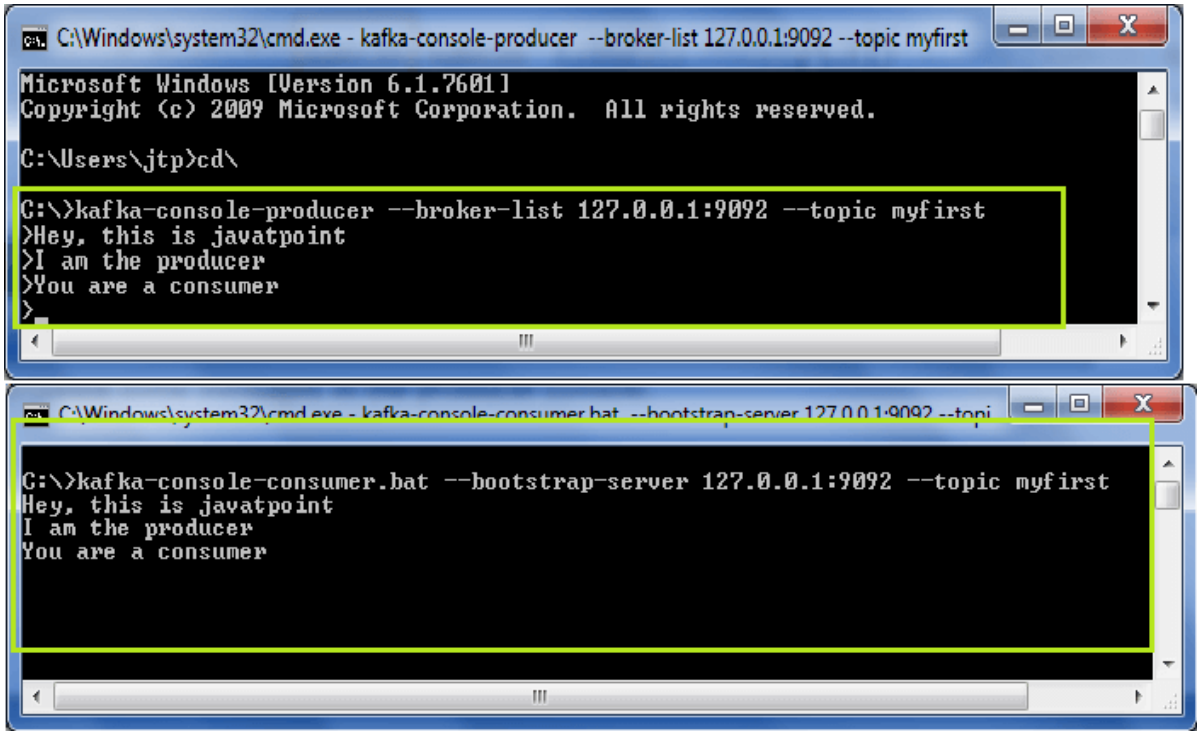
In the previous section, three messages were produced to this topic. But, in the above snapshot, 0 messages could be seen. It is because Apache Kafka does not read all the topics. A Kafka consumer will consume only those messages which are produced only when the consumer was in the active state. This can be categorized as a disadvantage of Apache Kafka.

Let's understand:

Open a new terminal. Launch the Kafka console producer. Keep both producer-consumer consoles together as seen below:



Now, produce some messages in the producer console. After doing so, press Ctrl+C and exit.



It is seen that all messages which are currently produced by the producer console are reflected in the consumer console. It is because the consumer is in an active state.

Reading whole messages

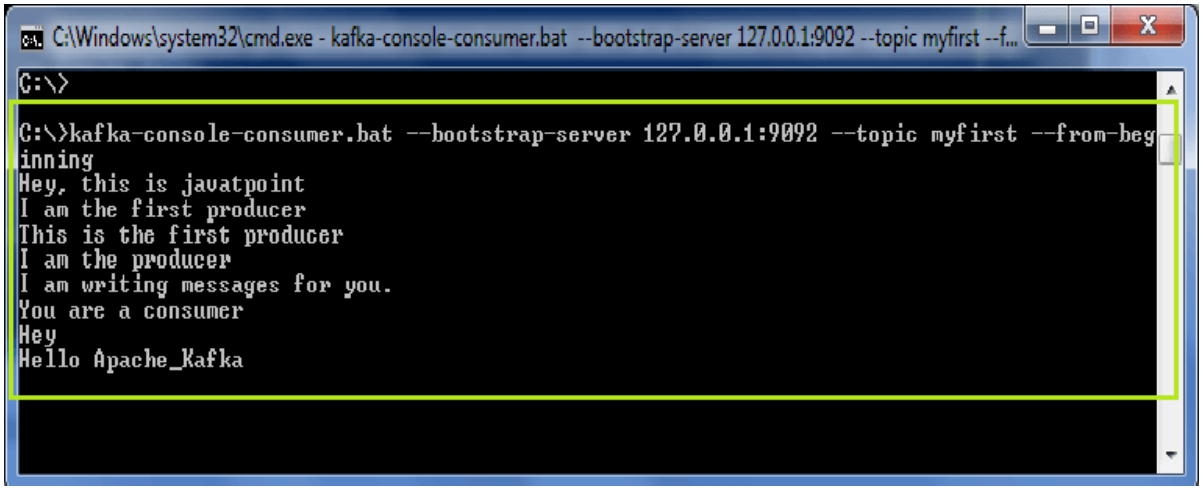
Apache Kafka allows to produce millions of messages. Sometimes, a consumer may require to read whole messages from a particular topic.

To do so, use '**--from-beginning**' command with the above kafka console consumer command as:

'kafka-console-consumer.bat --bootstrap-server 127.0.0.1:9092 --topic myfirst --from-beginning'.

This command tells the Kafka topic to allow the consumer to read all the messages from the beginning(i.e., from the time when the consumer was inactive).

For example,



In the above snapshot, it is clear that all messages are displayed from the beginning.

Note: The order of the messages is not the 'total'. It is because the sequence is at the partition level only(as studied in the Kafka introduction section).

For this topic 'myfirst', we had three partitions. So, if a user wishes to see the order, create a topic with a single partition value. It will display whole messages in a sequence.

After completing the message exchange process, press 'Ctrl+C' and stop.

So, several messages can be consumed either from the beginning or from that state when the user wants the consumer to read.

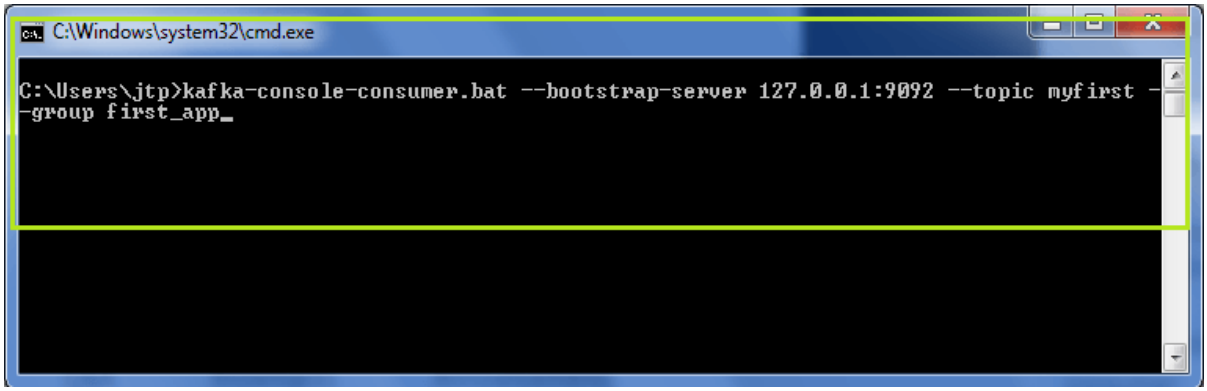
Kafka Consumer Group CLI

Generally, a Kafka consumer belongs to a particular consumer group. A consumer group basically represents the name of an application. In order to consume messages in a consumer group, '**-group**' command is used.

Let' see how consumers will consume messages from Kafka topics:

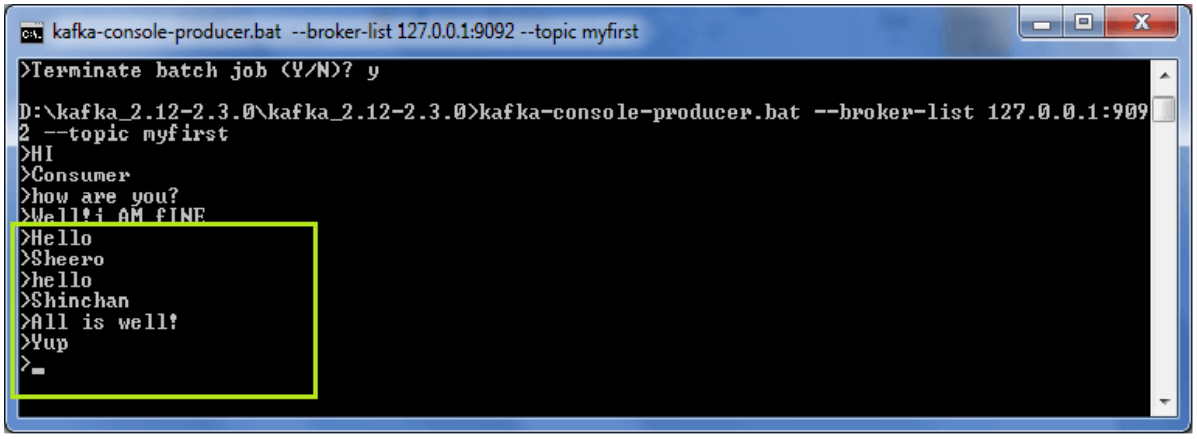
Step1: Open the Windows command prompt.

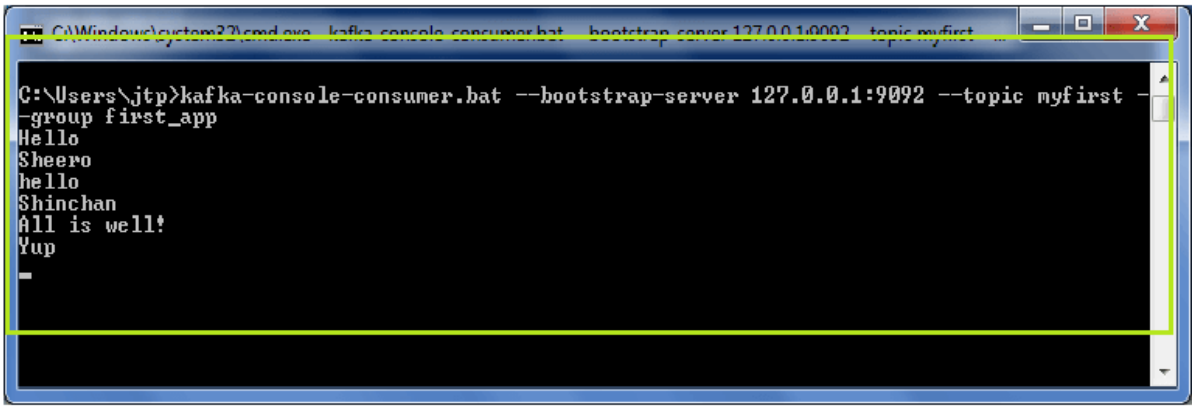
Step2: Use the '**--group**' command as: **'kafka-console-consumer --bootstrap-server localhost:9092 --topic --group <group_name>'**. Give some name to the group. Press enter.



In the above snapshot, the name of the group is '**first_app**'. It is seen that no messages are displayed because no new messages were produced to this topic. If '**--from-beginning**' command will be used, all the previous messages will be displayed.

Step3: To view some new messages, produce some instant messages from the producer console(as did in the previous section).

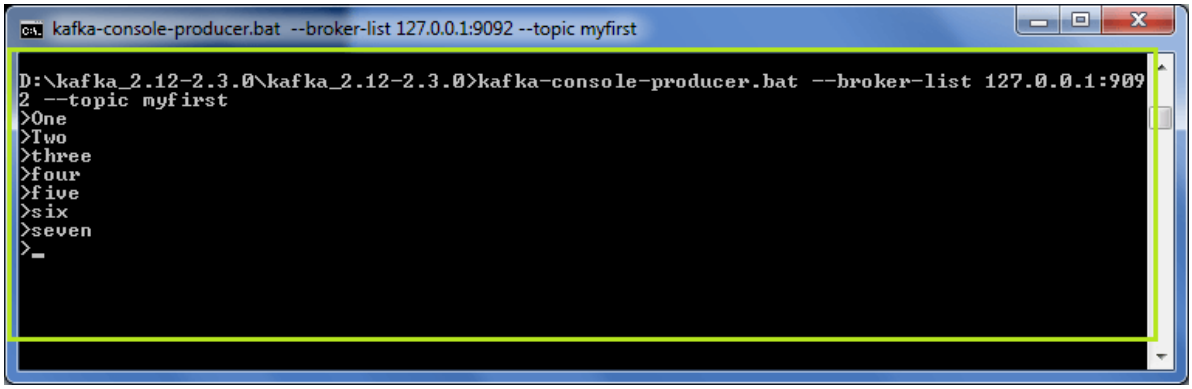




So, the new messages produced by the producer can be seen in the consumer's console.

Step4: But, it was a single consumer reading data in the group. Let's create more consumers to understand the power of a consumer group. For that, open a new terminal and type the exact same consumer command as:

'kafka-console-consumer.bat --bootstrap-server 127.0.0.1:9092 --topic <topic_name> --group <group_name>'.



In the above snapshot, it is clear that the producer is sending data to the Kafka topics. The two consumers are consuming the messages. Look at the sequence of the messages. As there were three partitions created for 'myfirst' topic(discussed earlier), so messages are split in that sequence only.

We can further create more consumers under the same group, and each consumer will consume the messages according to the number of partitions. Try yourself to understand better.

Note: The group id should be the same, then only the messages will be split between the consumers.

However, if any of the consumers is terminated, the partitions will be reassigned to the active consumers, and these active consumers will receive the messages.

So, in this way, various consumers in a consumer group consume the messages from the Kafka topics.

Consumer with Keys

When a producer has attached a key value with the data, it will get stored to that specified partition. If no key value is specified, the data will move to any partition. So, when a consumer reads the message with a key, it will be displayed null, if no key was specified. A '**print.key**' and a '**key.seperator**' sre required to consume messages from the Kafka topics. The command used is:

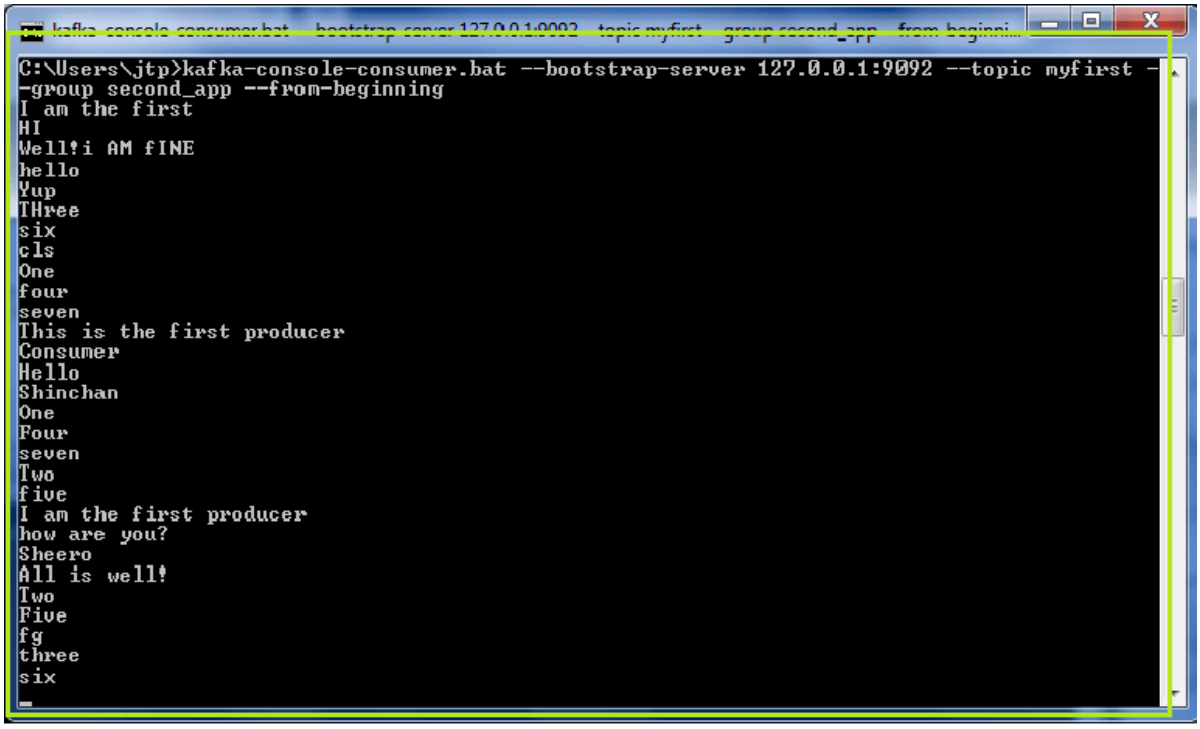
'kafka-console-consumer --bootstrap-server localhost:9092 --topic <topic_name> --from-beginning --property print.key=true --property key.seperator=,'

Using the above command, the consumer can read data with the specified keys.

More about Consumer Group

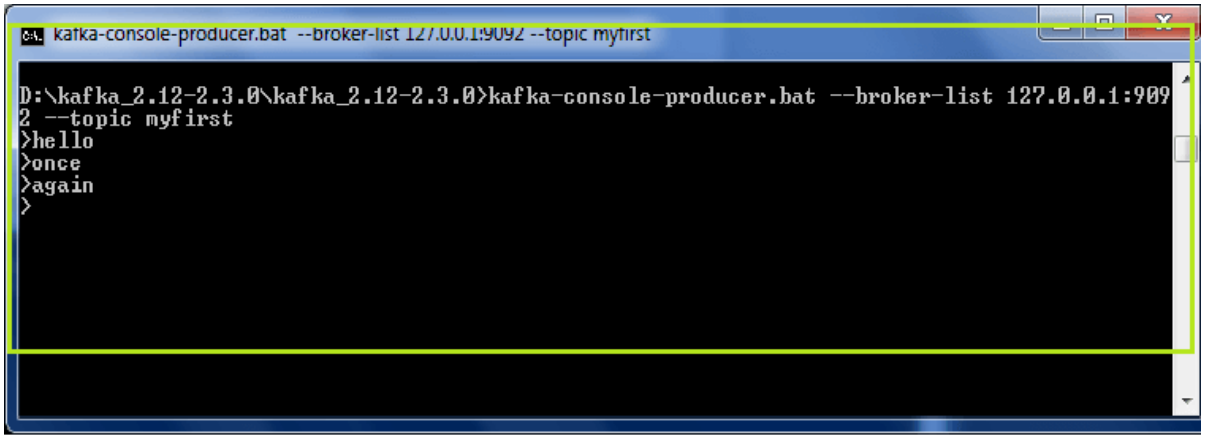
'--from-beginning' command

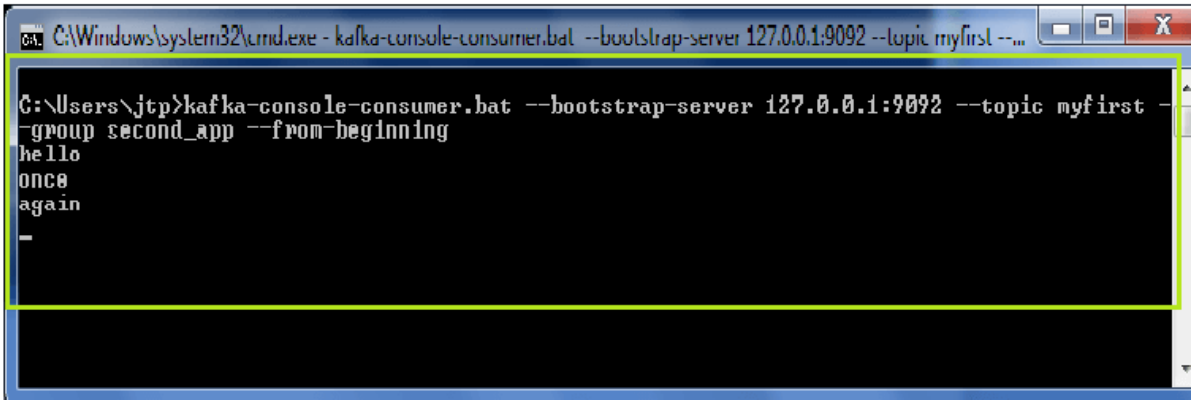
This command is used to read the messages from the starting(discussed earlier). Thus, using it in a consumer group will give the following output:



It can be noticed that a new consumer group 'second_app' is used to read the messages from the beginning. If one more time the same command will run, it will not display any output. It is because offsets are committed in Apache Kafka. So, once a consumer group has read all the until written messages, next time, it will read the new messages only.

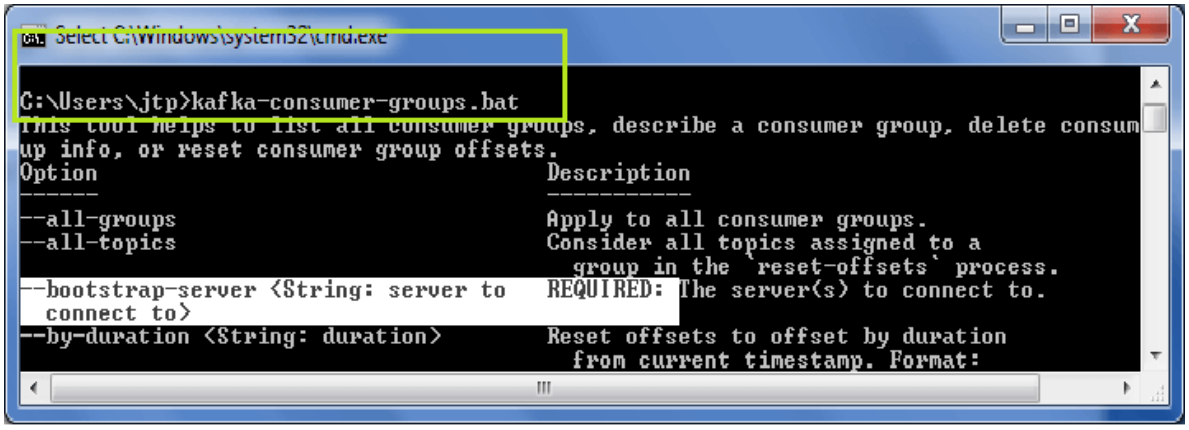
For example, in the below snapshot, when '**--from-beginning**' command is used again, only the new messages are read. It is because all the previous messages were consumed earlier only.





'kafka-consumer-groups' command

This command gives the whole documentation to list all the groups, describe the group, delete consumer info, or reset consumer group offsets.



It requires a bootstrap server for the clients to perform different functions on the consumer group.

Listing Consumer Groups

A '**--list**' command is used to list the number of consumer groups available in the Kafka Cluster. The command is used as:

'kafka-consumer-groups.bat --bootstrap-server localhost:9092 --list'.

A snapshot is shown below, there are three consumer groups present.

```
C:\Windows\system32\cmd.exe

C:\Users\jtp>kafka-consumer-groups.bat --bootstrap-server 127.0.0.1:9092 --list
first_app
first_appsix
second_app

C:\Users\jtp>_
```

Describing a Consumer Group

A '**--describe**' command is used to describe a consumer group. The command is used as:

'kafka-consumer-groups.bat --bootstrap-server localhost:9092 --describe group <group_name>'

```
C:\Windows\system32\cmd.exe

C:\Users\jtp>kafka-consumer-groups.bat --bootstrap-server 127.0.0.1:9092 --describe  --grou
p first_app

Consumer group 'first_app' has no active members.

GROUP           TOPIC           PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG
CONSUMER-ID     HOST            CLIENT-ID
first_app       myfirst         2          12              12              0
-               -               -
first_app       myfirst         1          11              11              0
-               -               -
first_app       myfirst         0          11              11              0
-               -               -

C:\Users\jtp>
```

This command describes whether any active consumer is present, the current offset value, lag value is 0 -indicates that the consumer has read all the data.

Resetting the Offsets

Offsets are committed in Apache Kafka. Therefore, if a user wants to read the messages again, it is required to reset the offsets value. '**Kafka-consumer-groups**' command offers an option to reset the offsets. Resetting the offset value means defining the point from where the user wants to read the messages again. It supports only one consumer group at a time, and there should be no active instances for the group.

While resetting the offsets, the user needs to choose three arguments:

1. An execution option
2. Reset Specifications
3. Scope

There are two executions options available:

'--dry-run': It is the default execution option. This option is used to plan those offsets that need to be reset.

'**--execute':** This option is used to update the offset values.

There are following reset specifications available:

'--to-datetime': It reset the offsets on the basis of the offset from datetime. The format used is: 'YYYY-MM-DDTHH:mm:SS.sss'.

'--to-earliest': It reset the offsets to the earliest offset.

'--to-latest': It reset the offsets to the latest offset.

'--shift-by': It reset the offsets by shifting the current offset value by 'n'. The value of 'n' can be positive or negative.

'--from-file': It resets the offsets to the values defined in the CSV file.

' --to-current': It reset the offsets to the current offset.

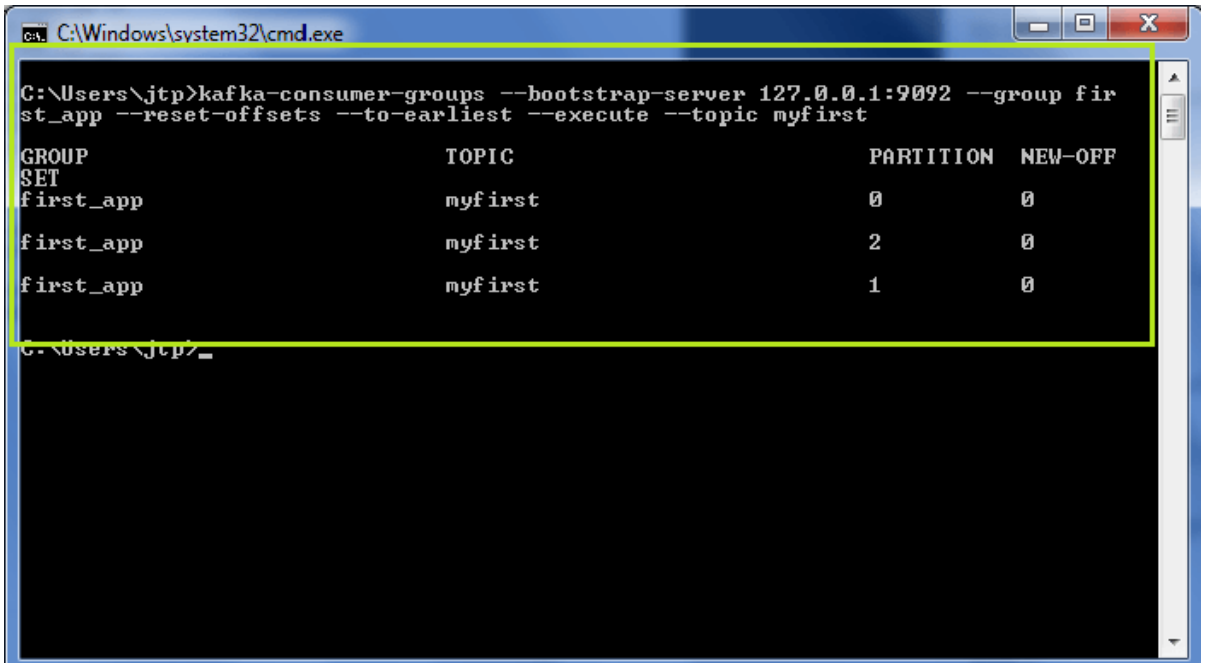
There are two scopes available to define:

'--all-topics': It reset the offset value for all the available topics within a group.

'--topics': It reset the offset value for the specified topics only. The user needs to specify the topic name for resetting the offset value.

Let's try and see:

1) Using '**--to-earliest**' command



In the above snapshot, the offsets are reset to the new offset as 0. It is because '**-to-earliest**' command is used, which has reset the offset value to 0.

2) Using '**--shift-by**' command

```
C:\Windows\system32\cmd.exe

C:\Users\jtp>kafka-consumer-groups --bootstrap-server 127.0.0.1:9092 --group second
_app --reset-offsets --shift-by 2 --execute --topic myfirst

GROUP                          TOPIC                          PARTITION  NEW-OFFSET

second_app                     myfirst                        0          2

second_app                     myfirst                        2          2

second_app                     myfirst                        1          2

C:\Users\jtp>_
```

```
C:\Windows\system32\cmd.exe

C:\Users\jtp>kafka-consumer-groups --bootstrap-server 127.0.0.1:9092 --group second
_app --reset-offsets --shift-by -1 --execute --topic myfirst

GROUP                          TOPIC                          PARTITION  NEW-OFFSET

second_app                     myfirst                        0          1

second_app                     myfirst                        2          1

second_app                     myfirst                        1          1

C:\Users\jtp>_
```

In the first snapshot, the offset value is shifted from '0' to '+2'. In the second one, the offset value is shifted from '2' to '-1'.